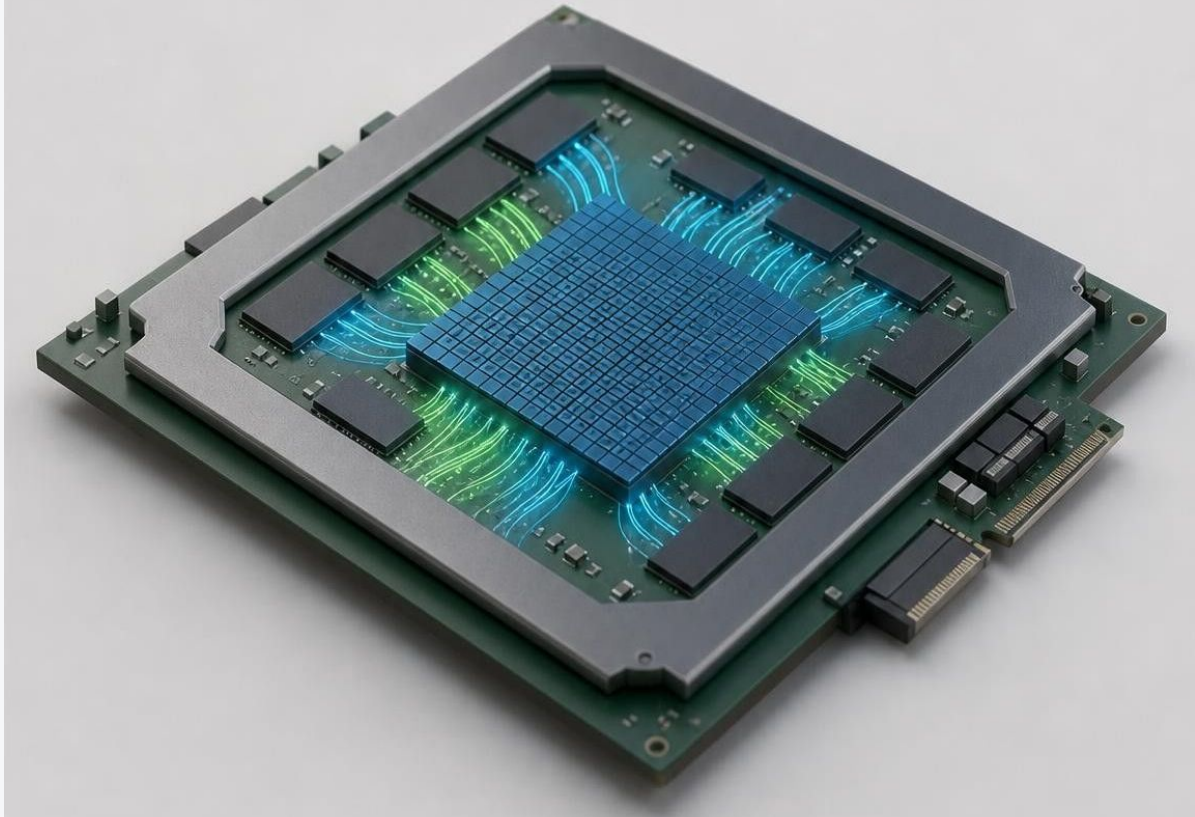
Why is my LLM slow?

Memory bandwidth vs compute, explained from scratch

The model fits, loads and answers, but slowly, or with a long pause before the first word. Most people blame the math. Usually wrong: the math units spend most of their time **waiting for numbers to move** from GPU memory to the cores.

By the end, we can estimate tokens per second by hand.

BANDWIDTH COMPUTE TOKENS/S



What we will cover

01

The two limits

"Does it fit?" vs "is it fast?", compute and bandwidth, real numbers for common GPUs

02

What one token costs

Math per token, bytes moved per token, why moving bytes takes far longer, memory-bound vs compute-bound

03

Estimating speed by hand

The tokens-per-second formula, one model on four GPUs

04

Prefill vs decode

Why reading the prompt is fast but writing is slow

05

Making it faster

Quantization, batching many users, why long conversations slow down

06

Case studies

Real case studies and a cheat sheet

”Does it fit?” and ”Is it fast?” are different questions

- GPU memory has a **size** (GB) and a **speed**: bytes per second from VRAM to the cores, called **memory bandwidth** (GB/s or TB/s)
- **Memory size** decides whether the model can run; **bandwidth** decides how fast it runs once it fits

GPU A

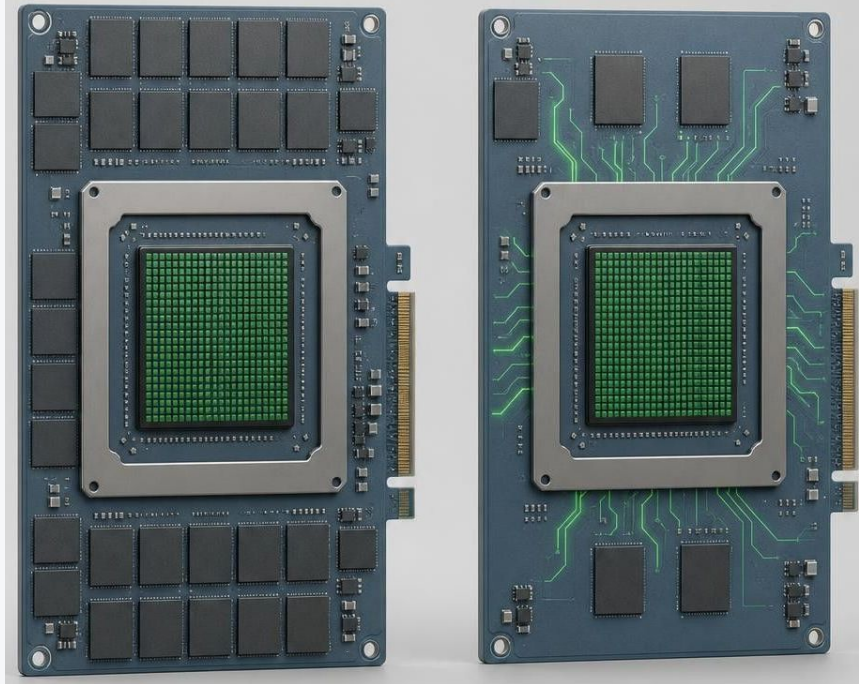
48 GB · 900 GB/s

GPU B

24 GB · 1,000 GB/s

A 30 GB model runs only on A. A 10 GB model runs on both, and B is slightly faster with half the memory.

Bigger memory does not mean a faster model. This deck focuses only on speed.





The two limits on every GPU: compute and bandwidth

Compute

Math per second in FLOPs per second; a FLOP is one multiplication or addition. A modern data-centre GPU: ~1,000 TFLOPS in BF16 (tera = trillion)

Memory bandwidth

Bytes per second from VRAM to the cores; the same GPU moves ~3.35 TB/s. Cores can only do math on numbers that have arrived

1	2	3	4
Numbers in VRAM	**Travel to cores**	**Cores do math**	**Results written back**
	bandwidth limit	compute limit	

Very important: slow arrival leaves cores idle however fast they are; huge math makes the cores the bottleneck.

The question for any work: **which limit do we hit first?**

Real numbers for common GPUs

Rough figures from makers' published specs, rounded for easy math.

GPU	Memory size	Bandwidth	Compute (FP16/BF16)	Math steps per byte moved
NVIDIA T4 (Colab)	16 GB	~320 GB/s	~65 TFLOPS	~200
NVIDIA RTX 4090	24 GB	~1,000 GB/s	~165 TFLOPS	~165
NVIDIA A100 80GB	80 GB	~2,000 GB/s	~312 TFLOPS	~156
NVIDIA H100 (SXM)	80 GB	~3,350 GB/s	~990 TFLOPS	~295

Compute ÷ bandwidth = math steps per byte

To keep the GPU fully busy, each byte from VRAM must be used in **about 150 to 300 math steps**; otherwise the cores finish early and wait. Keep this number in mind.

How much math does one token need?

Each new token passes through every layer; each weight takes about two math steps: one multiplication (input × weight) and one addition (to a running total).

One token ≈ 2 FLOPs per parameter

- Llama 3 8B ≈ 16 billion FLOPs · 13B ≈ 26 billion · Llama 3 70B ≈ 140 billion
- Llama 3 8B on an H100 (~1,000 trillion FLOPs/s): 16 billion ÷ 1,000 trillion = **0.016 ms**

Real chatbots give about 30 to 200 tokens per second for one user, so something else is slowing it down.

IF MATH WERE THE ONLY LIMIT

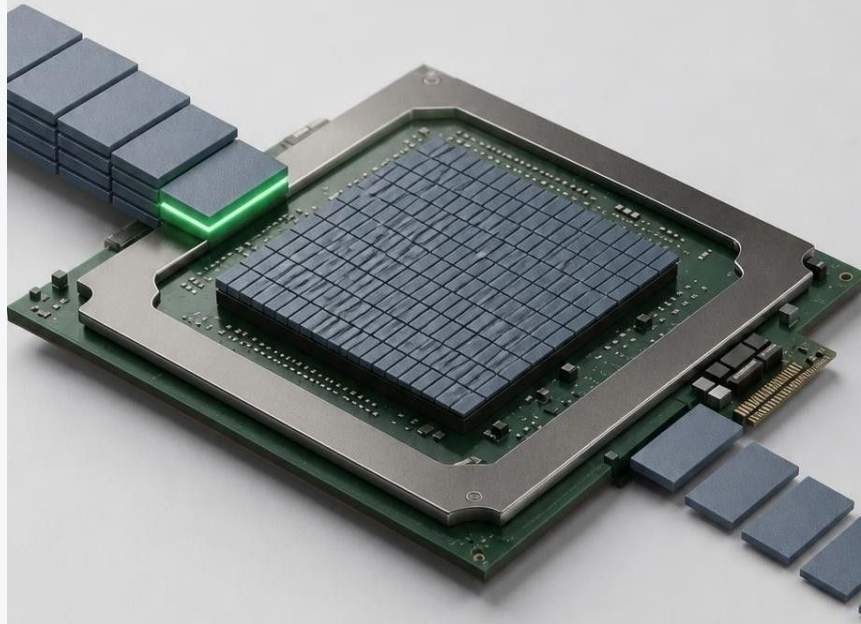
60,000

tokens per second for one user

Every token moves the whole model through memory

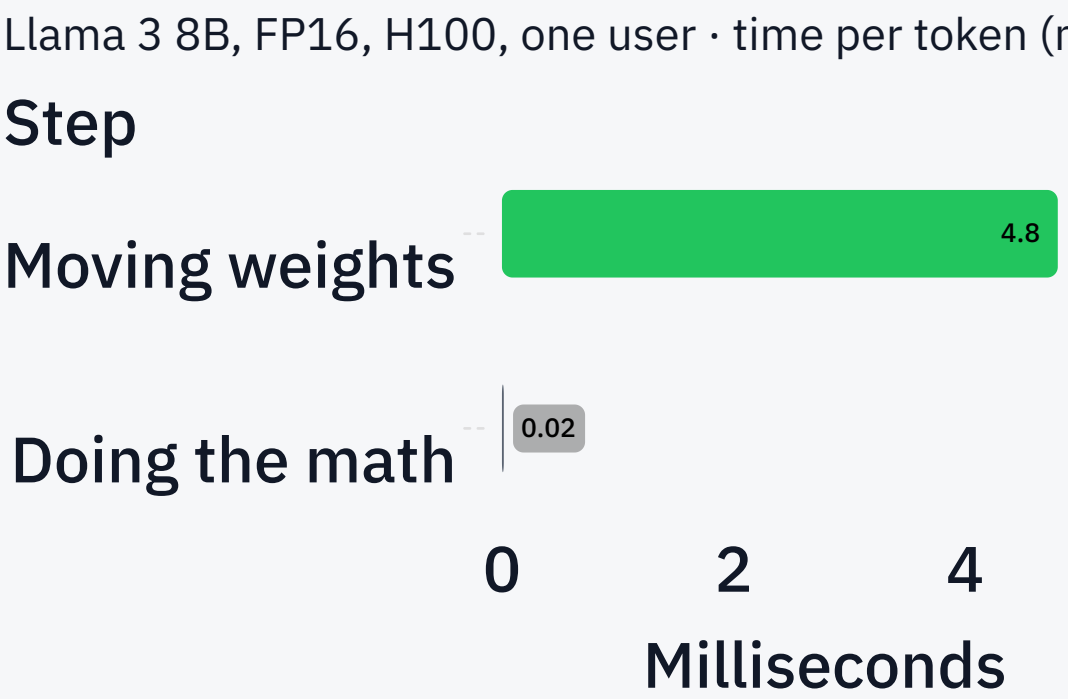
- **Key fact:** one token uses every weight once, so every weight travels from VRAM to the cores once per token
- Cores hold only a few tens of MB of fast storage; Llama 3 8B has 16 GB of FP16 weights, hundreds of times more
- Layer 1's weights are brought in, used, then pushed out for layer 2; by the last layer they're long gone, so token 1, 2, 3… each read layer 1 → layer 2 → … → layer 32, every time

Model	Precision	Bytes moved per token
Llama 3 8B	FP16	**~16 GB**
Llama 3 8B	INT8	**~8 GB**
Llama 3 8B	INT4	**~4 GB**
Llama 3 70B	FP16	**~140 GB**



Moving bytes takes about 300× longer than the math

Llama 3 8B, FP16, H100, one user · time per token (ms)



Math: 16 billion FLOPs ÷ 990 trillion FLOPs/s ≈ 0.016 ms ·
Moving weights: 16 GB ÷ 3,350 GB/s ≈ 4.8 ms

4.8 ms

vs 0.016 ms of math

- The cores finish their math in a tiny moment, then wait: **more than 99% of the time, the math units are idle**
- That's why a GPU with more TFLOPS often does almost nothing for single-user chatbot speed

For one user, text-generation speed is set almost entirely by memory bandwidth.

Memory-bound vs compute-bound

Memory-bound

Waiting for data; faster math doesn't help, only more bandwidth or fewer bytes

Compute-bound

Data arrives fast enough, cores stay busy; faster math helps

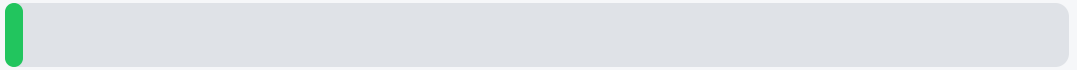
Arithmetic intensity = math steps done ÷ bytes moved

One token, one user, FP16: 2 FLOPs ÷ 2 bytes per parameter = **1 math step per byte**.

H100 can handle **~295 per byte**



Our work needs **1 per byte**



Cores run out of work almost instantly → they wait for bytes → memory-bound.

Rule: intensity lower than the GPU's ratio = memory-bound; higher = compute-bound. One token at a time for one user is one of the most memory-bound jobs a GPU can get.

Estimating tokens per second by hand

Max tokens/s ≈ Memory bandwidth ÷ Model size in memory

- Llama 3 8B FP16 (16 GB) on an H100 (3,350 GB/s): 3,350 ÷ 16 ≈ 210 tokens/s, the ceiling
- In practice expect 60–80%: the GPU never reaches 100% of listed bandwidth, the KV cache is also read every token, picking the next token takes small steps, and software adds delays
- Realistic: about **130 to 170 tokens/s**

A useful skill: from any model size and GPU bandwidth, predict single-user speed without running anything.

CEILING · 8B ON H100

210

tokens/s = 3,350 ÷ 16

Case study 1: the same model on four GPUs

A team runs Llama 3 8B in INT8 (8 GB, fits on all four). Formula: bandwidth ÷ 8 GB.

GPU	Bandwidth	Ceiling (tokens/s)	Realistic (~70%)
T4	320 GB/s	~40	~28
RTX 4090	1,000 GB/s	~125	~88
A100 80GB	2,000 GB/s	~250	~175
H100	3,350 GB/s	~420	~290

- Speed follows bandwidth almost exactly: H100 has ~10× the T4's bandwidth and is ~10× faster
- A100 is ~2× the 4090: it also has ~2× the compute, but the reason is its ~2× bandwidth
- T4 at ~28 tokens/s beats reading speed: fine for learning, sluggish for customers

Lesson: for a few users, check bandwidth first, then memory size; compute matters much less.

Prefill vs decode: the two phases of every answer

PREFILL · READING THE PROMPT

Compute-bound

1 2 3

2,000 prompt tokens, known

Weights read once

Used for all 2,000

The model already knows all, say, 2,000 tokens, so it processes them together → very high arithmetic intensity. Each byte feeds thousands of math steps, far above the GPU's ~150–300, so cores stay fully busy.

DECODE · WRITING THE ANSWER

Memory-bound

1 2 3

Token 1

Token 2

Token 3…

One token at a time, since token 2 depends on token 1. Each token needs its own full trip through the weights → intensity ≈ 1.

Same model, same GPU, completely different behaviour in the two phases.

Case study 2: why the chatbot pauses, then types

A user pastes a 2,000-token document into Llama 3 8B (FP16, H100) and asks for a 500-token summary: a short pause, then steady words.

PREFILL

~0.05–0.1 s

2,000 × 16 billion = 32 trillion FLOPs ÷ ~990 TFLOPS ≈ 0.03 s at best

DECODE

3.3 s

500 tokens ÷ ~150 tokens/s

It read 4× more tokens than it wrote, but reading took about 30× less time.

Time to First Token (TTFT): wait before the first word, mostly prefill; a longer prompt raises it.

Time Per Output Token (TPOT): time per next word, the decode speed; a longer answer raises total time.

Asking for a short answer is often the easiest way to get a faster response.

Quantization makes models faster, not just smaller

Tokens/s ≈ bandwidth ÷ bytes moved per token, so fewer bytes per weight = more speed

Llama 3 8B on an RTX 4090 (~1,000 GB/s)

Precision	Bytes moved per token	Ceiling (tokens/s)
FP16	16 GB	~62
INT8	8 GB	~125
INT4	4 GB	~250

- Real gains are smaller: 4-bit weights must be converted back to FP16 inside the core, the KV cache usually stays FP16, and fixed software delays don't shrink
- In practice FP16 → INT4 gives about **2 to 3×** faster decode, not 4×
- The conversion adds math, but in decode the cores were idle anyway: adding a bit of math to save many bytes is a good trade

Batching: read the weights once, use them for everyone

WITHOUT BATCHING · 1 USER

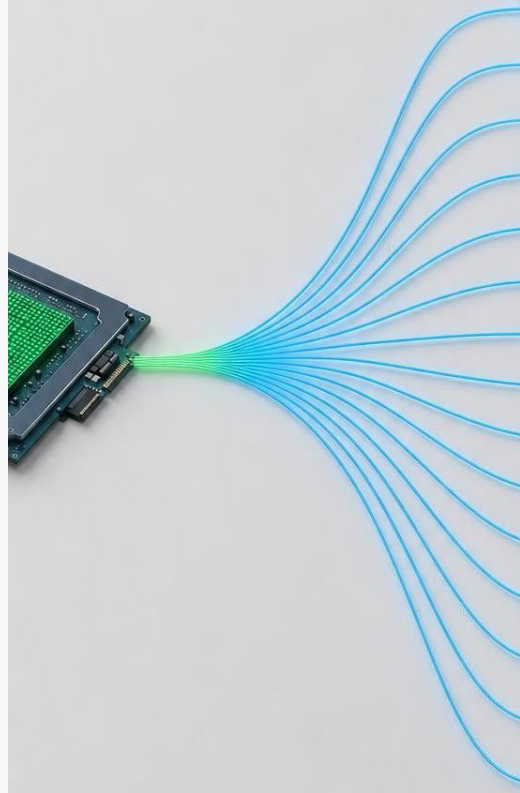
Each decode step reads all weights to make one token, cores mostly idle

All weights → one token

WITH BATCHING · 32 USERS

The server reads the weights once and makes the next token for all 32 in one step

Weights once → 32 tokens



- Bytes per step: almost the same · Math per step: 32× more · Arithmetic intensity: ~1 → ~32
- Time per step barely changes, since the weights take the same time to arrive

32×

With batching, the GPU makes 32 tokens in about the time it used to make 1. That's why serving many users at once is far cheaper per user, and why big AI companies can price chatbots low.

Case study 3: serving 1, 32, and 256 users

Llama 3 8B FP16 on one H100, ~1,000-token conversations, KV cache ~0.13 MB per token (~0.13 GB per user). Ideal estimates to show the pattern.

Users in batch	Bytes read per step	Time per step	Speed per user	Total tokens/s
1	16 GB weights + 0.13 GB cache	~4.8 ms	~210	~210
32	16 GB + 4 GB of caches	~6 ms	~165	~5,300
256	16 GB + 33 GB of caches	~15 ms	~68	**~17,500**

17,500

total tokens/s at 256 users, over 80× on the same GPU

- Each user slows from 210 to ~68 tokens/s, still far faster than reading
- At 256 users caches (33 GB) are twice the weights; each cache is per user, not shared, so gains slow with longer chats
- **Lesson:** batch size is a business choice, faster per user vs cheaper per token

Long conversations slow the model down

Every decode step reads all the weights (fixed) plus that user's whole KV cache (growing). Llama 2 7B, H100, one user, FP16, no GQA (~0.5 MB per token):

Conversation length	Weights	KV cache	Total read per token	Ceiling (tokens/s)
1,000 tokens	14 GB	0.5 GB	14.5 GB	~230
8,000 tokens	14 GB	4 GB	18 GB	~185
32,000 tokens	14 GB	16 GB	30 GB	~110

At 32,000 tokens the model is about **half as fast**, only from the longer conversation

Llama 3 8B with GQA: ~4 GB cache at 32,000 tokens, ~20 GB total, ceiling stays around **165 tokens/s**

Lesson: GQA saves memory and keeps long conversations fast



Case study 4: big models and Mixture of Experts

LLAMA 3 70B · FP16 · TWO H100S

48

140 GB split; each GPU reads its half at once, ~6,700 GB/s combined → 6,700 ÷ 140 ≈ 48 tokens/s

LLAMA 3 70B · INT4 · ONE H100

95

35 GB → 3,350 ÷ 35 ≈ 95 tokens/s. One GPU in INT4 beats two in FP16: 4× fewer bytes, no delay from GPUs exchanging results every layer

MIXTRAL 8X7B

~26 GB

~47B parameters in memory, ~13B used per token; decode reads only the experts used: ~26 GB per token in FP16, not 93 GB, so roughly 13B-model speed with a bigger model's knowledge capacity

Catch: in a big batch, users' tokens pick different experts, so most get read anyway; the advantage is strongest with few users.

Case study 5: why a chatbot types at the speed it does

Reading speed

Adults read ~200–300 words per minute, about 5–7 tokens per second

GPU speed

A mid-sized model on a modern GPU reaches 100–300 tokens/s for one user

Cost

One user at 210 tokens/s vs 256 users at ~68 each, over 80× more total output (Case study 3)

Streaming

Each token is shown as soon as it's made, so a 10-second answer starts reading in under a second

Small batch: each user very fast → few users per GPU → high cost per user.
Large batch: each user still fast → many users per GPU → low cost per user.

The typing speed is a deliberate choice: comfortably faster than reading, with spare capacity serving more people. A "fast mode" at a higher price just means fewer users share each GPU.



Cheat sheet: the whole deck on one slide

- **Two limits:** compute (TFLOPS) and bandwidth (GB/s or TB/s); fitting depends on memory size, speed mostly on bandwidth
- **One token costs:** ~2 FLOPs per parameter, and ~the full weight size in bytes moved
- **Single-user decode is memory-bound:** cores idle over 99% of the time
- **Prefill** reads the prompt at once, compute-bound and fast; **decode** writes one token at a time, memory-bound and slow

Max tokens/s ≈ bandwidth ÷ (weight size + KV cache size); expect 60–80%

Remember: when an LLM is slow at writing, the problem is almost never the math. **It is the bytes.**

Change	Why it helps	What we give up
Quantize the weights	Fewer bytes moved per token	A little accuracy
More-bandwidth GPU	Bytes arrive faster	Higher cost
Model with GQA	Smaller KV cache read every token	Almost nothing
Shorter conversations	Smaller KV cache read every token	Less context
Ask for shorter answers	Fewer decode steps	Less detail
Batch many users	Weights read once, used for everyone	Each user a bit slower